

Asignatura: Despliegue de Aplicaciones Web

UD 6: Documentación, sistemas de control de versiones y de integración continua

Tarea 1:

Implementación y documentación de un sistema de control de versiones con Git

Índice

1. [Introducción](#)
2. [Instalación de Git en GNU/Linux](#)
3. [Configuración inicial de Git](#)
4. [Creación de un repositorio local y primer control de versiones](#)
5. [Conexión del repositorio local con GitHub](#)
6. [Control de cambios: modificación, commit y actualización remota](#)
7. [Descarga de cambios remotos mediante git pull](#)
8. [Accesibilidad y seguridad del repositorio](#)
9. [Documentación del sistema de control de versiones](#)
10. [Gestión de flujos de trabajo](#)
11. [Conclusión](#)
12. [Bibliografía y recursos](#)

1. Introducción

En esta práctica se trabaja la implementación y documentación de Git como sistema de control de versiones dentro de un entorno GNU/Linux. Git es una herramienta fundamental en el desarrollo de aplicaciones web, ya que permite registrar los cambios realizados en los archivos de un proyecto, recuperar versiones anteriores y mantener un historial claro de la evolución del código.

Para realizar la práctica se ha utilizado una máquina virtual con Ubuntu en VirtualBox. En este entorno se ha instalado Git mediante el gestor de paquetes de la distribución, se ha configurado la identidad del usuario y se ha creado un repositorio local de prueba llamado proyecto-git-ud6-ddaw. Sobre este repositorio se han realizado operaciones básicas de control de versiones, como añadir archivos, confirmar cambios mediante commits, revisar el estado del repositorio y consultar el historial.

Además, el repositorio local se ha conectado con GitHub para comprobar el trabajo con un repositorio remoto. Para ello se ha utilizado una conexión mediante clave SSH, lo que permite autenticar el equipo de forma segura sin introducir la contraseña de la cuenta en cada operación. También se ha comprobado el funcionamiento de comandos como git push y git pull, que permiten enviar cambios al repositorio remoto y descargar modificaciones realizadas desde GitHub.

A lo largo del documento se explican los pasos realizados, los comandos utilizados y la utilidad de cada operación. También se incluyen aspectos relacionados con la accesibilidad y seguridad del repositorio, así como los archivos y flujos de trabajo recomendados para documentar correctamente un sistema de control de versiones.

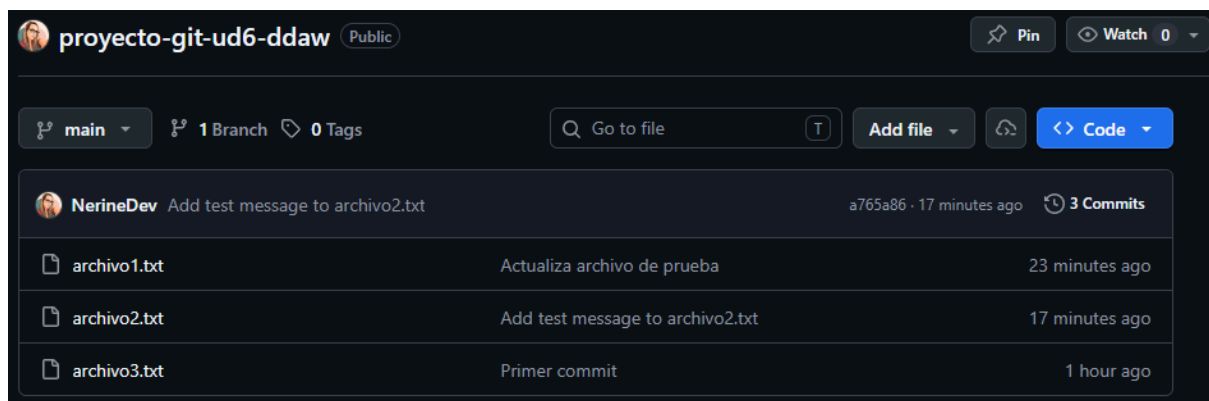


Figura 1. Repositorio de GitHub utilizado como repositorio remoto para la práctica.

2. Instalación de Git en GNU/Linux

Para comenzar la práctica fue necesario instalar Git en el sistema GNU/Linux utilizado. En este caso se trabajó con Ubuntu, por lo que la instalación se realizó mediante APT, el gestor de paquetes propio de esta distribución. Antes de instalar el programa, se actualizó la lista de paquetes disponibles para asegurar que el sistema consultara la información más reciente de los repositorios.

El primer comando utilizado fue:

```
sudo apt update
```

Este comando no instala Git directamente, sino que actualiza la información de los paquetes disponibles en los repositorios configurados en el sistema. Después de finalizar este proceso, se ejecutó la instalación de Git mediante el siguiente comando:

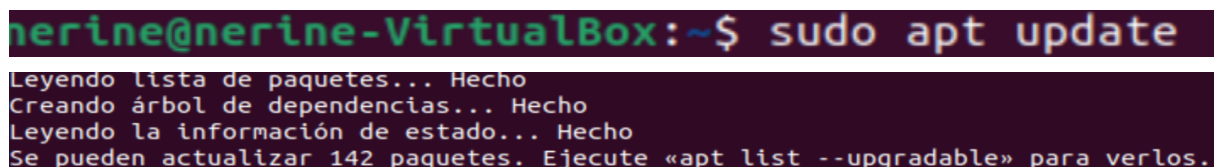
```
sudo apt install git
```

Durante la instalación, el sistema descarga e instala Git junto con los componentes necesarios para su funcionamiento. El uso del gestor de paquetes de la distribución resulta recomendable porque permite instalar la herramienta desde fuentes oficiales y mantenerla actualizada mediante las actualizaciones normales del sistema.

Una vez terminada la instalación, se comprobó que Git estaba disponible correctamente utilizando el comando:

```
git --version
```

La respuesta del sistema mostró la versión instalada de Git, lo que confirma que la instalación se completó correctamente y que la herramienta ya podía utilizarse desde la terminal.



```
nerine@nerine-VirtualBox:~$ sudo apt update
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 142 paquetes. Ejecute «apt list --upgradable» para verlos.
```

Figura 2. Actualización de los repositorios de Ubuntu mediante sudo apt update.

```
nerine@nerine-VirtualBox:~$ sudo apt install git
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
git-man liberror-perl
```

Figura 3. Instalación de Git en Ubuntu mediante el gestor de paquetes APT.

```
nerine@nerine-VirtualBox:~$ git --version
git version 2.34.1
```

Figura 4. Verificación de la instalación de Git mediante el comando git --version.

3. Configuración inicial de Git.

Después de instalar Git, se realizó la configuración inicial del usuario. Esta configuración es necesaria porque Git registra en cada commit quién ha realizado los cambios. De esta forma, en un proyecto individual o colaborativo es posible identificar el autor de cada modificación dentro del historial del repositorio.

Para configurar el nombre de usuario se utilizó el siguiente comando:

```
git config --global user.name "NerineDev"
```

A continuación, se configuró el correo electrónico asociado a la cuenta de GitHub. En este caso se utilizó la dirección privada proporcionada por GitHub, ya que permite asociar los commits a la cuenta sin mostrar el correo personal de forma pública:

```
git config --global user.email
"117679910+NerineDev@users.noreply.github.com"
```

```
nerine@nerine-VirtualBox:~$ git config --global user.email "117679910+NerineDev@
users.noreply.github.com"
nerine@nerine-VirtualBox:~$ git config --global user.name "NerineDev"
```

Figura 5. Configuración global del nombre de usuario y correo electrónico en Git.

También se activó la salida en color en la terminal mediante el comando:

```
git config --global color.ui auto
```

```
nerine@nerine-VirtualBox:~$ git config --global core.editor "code --wait"
nerine@nerine-VirtualBox:~$ git config --global color.ui auto
```

Figura 6. Activación de salida en color en la terminal

Esta opción facilita la lectura de los resultados de Git, ya que permite diferenciar visualmente archivos modificados, archivos preparados para commit y otros estados del repositorio.

Finalmente, se comprobó la configuración aplicada mediante:

```
git config --global --list
```

La salida del comando permitió verificar que el nombre de usuario, el correo electrónico y la opción de color habían quedado guardados correctamente en la configuración global de Git.

```
nerine@nerine-VirtualBox:~$ git config --global --list
user.name=NerineDev
user.email=117679910+NerineDev@users.noreply.github.com
```

Figura 7. Comprobación de la configuración inicial de Git mediante git config --global --list.

4. Creación de un repositorio local y primer control de versiones.

Una vez configurado Git, se creó un repositorio local de prueba para comprobar el funcionamiento básico del control de versiones. Para ello, primero se creó una carpeta de trabajo llamada proyecto-git-ud6-ddaw y se accedió a ella desde la terminal.

Los comandos utilizados fueron:

```
mkdir proyecto-git-ud6-ddaw
cd proyecto-git-ud6-ddaw
```

A continuación, se inicializó el repositorio mediante el comando:

```
git init
```

```
nerine@nerine-VirtualBox:~$ mkdir proyecto-git-ud6-ddaw
nerine@nerine-VirtualBox:~$ cd proyecto-git-ud6-ddaw
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git init
Iniciado repositorio Git vacío en /home/nerine/proyecto-git-ud6-ddaw/.git/
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$
```

Figura 8. Inicialización del repositorio local mediante git init.

Este comando crea dentro de la carpeta un directorio oculto llamado `.git`, donde Git almacena la información necesaria para controlar el historial del proyecto. Al tratarse de un repositorio nuevo, Git mostró un aviso sobre el nombre inicial de la rama. Para utilizar la convención actual más habitual, se renombró la rama principal como `main` mediante:

```
git branch -M main
```

Después se crearon tres archivos de texto para utilizarlos como contenido inicial del repositorio:

```
echo "Archivo de prueba 1" > archivo1.txt
echo "Archivo de prueba 2" > archivo2.txt
echo "Archivo de prueba 3" > archivo3.txt
```



```
Inicializado repositorio Git vacio en /home/nerine/proyecto-git-ud6-ddaw/.git/
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git branch -m main
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ echo "Archivo test 1" > archiv
o1.txt
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ echo "Archivo test 2" > archiv
o2.txt
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ echo "Archivo test 3" > archiv
o3.txt
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ ls
archivo1.txt  archivo2.txt  archivo3.txt
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git status
En la rama main

No hay commits todavía

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    archivo1.txt
    archivo2.txt
    archivo3.txt

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa
"git add" para hacerles seguimiento)
```

Figura 9. Estado del repositorio antes de añadir los archivos al área de preparación.

Al ejecutar `git status`, Git mostró estos archivos como archivos sin seguimiento. Esto significa que existen en la carpeta del proyecto, pero todavía no forman parte del historial del repositorio.

Para incluirlos en el próximo commit, se utilizó:

```
git add .
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git add .
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git status
En la rama main

No hay commits todavía

Cambios a ser confirmados:
  (usa "git rm --cached <archivo>..." para sacar del área de stage)
nuevos archivos: archivo1.txt
nuevos archivos: archivo2.txt
nuevos archivos: archivo3.txt
```

Figura 10. Estado del repositorio tras añadir los archivos al área de preparación.

Este comando añade los archivos al área de preparación, también llamada staging area. A partir de ese momento, los archivos quedan listos para ser confirmados. Finalmente, se realizó el primer commit con el comando:

```
git commit -m "Primer commit"
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git commit -m "Primer commit"
[main (commit-raíz) e56408c] Primer commit
3 files changed, 3 insertions(+)
create mode 100644 archivo1.txt
create mode 100644 archivo2.txt
create mode 100644 archivo3.txt
```

Figura 11. Confirmación del primer commit del proyecto.

El commit guarda una primera versión del proyecto dentro del historial de Git. Para comprobarlo, se utilizó:

```
git log --oneline
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git log --oneline
e56408c (HEAD -> main) Primer commit
```

Figura 12. Consulta del historial del repositorio mediante git log --oneline.

Este comando permite ver el historial de commits de forma resumida, mostrando el identificador de cada commit y el mensaje asociado.

5. Conexión del repositorio local con GitHub.

Después de crear el repositorio local y realizar el primer commit, se conectó el proyecto con un repositorio remoto en GitHub. El repositorio remoto permite almacenar una copia del proyecto en la nube, acceder al código desde otros equipos y facilitar el trabajo colaborativo si participan varios desarrolladores.

En primer lugar, se creó en GitHub un repositorio llamado proyecto-git-ud6-ddaw. A continuación, se añadió la dirección del repositorio remoto al proyecto local mediante el comando:


```
git remote add origin https://github.com/NerineDev/proyecto-git-ud6-ddaw.git
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git remote add origin https://github.com/NerineDev/proyecto-git-ud6-ddaw.git
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git remote -v
origin  https://github.com/NerineDev/proyecto-git-ud6-ddaw.git (fetch)
origin  https://github.com/NerineDev/proyecto-git-ud6-ddaw.git (push)
```

Figura 13. Comando para agregar el repositorio remoto al proyecto local

Sin embargo, para mejorar la seguridad y evitar introducir la contraseña o un token personal en cada operación, se configuró el acceso mediante clave SSH. Para ello, se generó una clave en Ubuntu y se añadió la clave pública a la cuenta de GitHub. Después se comprobó la autenticación mediante:

```
ssh -T git@github.com
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ cat /home/nerine/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDnzdBxWVn7q3ueYq4V/lbkRiYWx1q789KWcnTHp+Zj
dSLKRHGigIlW+xl52Qci0/eBpFsGCpAD1GhjXXp260Ncb600o+0ghEGghaSBY23WmaihtHwDedBAs8m
5G1fBkEs3F7MgPuSy6mFm0lmbGUpLBllZ6DZUvi2H0u83CKUM8f04afPMNEbh3G8NnBcq8zF/3VywAAI
xhvx20XNqoB4MNUiQxpXnh1MC6ENls0xfV91oKq0iOXHHmD/2uI/2CvFKyx/uP63Y2RIiVhIgYh0a4o
KijH7rFR+CJ7wERYQsF4Bxjh6MN95a726UvEUG5LzJWfKbXhIsUjaJD1kONkX6F80nGvJv/KH5t8ofxN
vKDFeX+CLpErcUxMIRjHoB39HFAqbR/+sW2QpHirX3s7TpgVDZpIqhMhTbvbc+UstRxBsxjVsLVUY8q4
Zo2RW0mHW5ygw2Du1g0paPHoZPJdyJ822Qj2I60Fj7RGkf8RqpAiLR2rWuOXf13mRuZQoZ0= nerine@
nerine-VirtualBox
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi NerineDev! You've successfully authenticated, but GitHub does not provide shell access.
```

Figura 14. Comprobación de la autenticación con GitHub mediante clave SSH.

La respuesta de GitHub confirmó que la autenticación se había realizado correctamente. Este paso demuestra que el equipo local quedó autorizado para comunicarse con GitHub mediante SSH.

Una vez configurada la clave SSH, se cambió la dirección remota del repositorio para utilizar el formato SSH:

```
git remote set-url origin
git@github.com:NerineDev/proyecto-git-ud6-ddaw.git
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git remote set-url origin git@github.com:NerineDev/proyecto-git-ud6-ddaw.git
```

Figura 15. Configuración del repositorio remoto mediante SSH.

Después se comprobó la configuración del repositorio remoto con:

```
git remote -v
```



```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git remote -v
origin  https://github.com/NerineDev/proyecto-ud6-ddaw.git (fetch)
origin  https://github.com/NerineDev/proyecto-ud6-ddaw.git (push)
```

Figura 16. Comprobación del repositorio.

Finalmente, se envió la rama main al repositorio remoto mediante:

```
git push -u origin main
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git push -u origin main
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (5/5), 347 bytes | 347.00 KiB/s, listo.
Total 5 (delta 0), reusados 0 (delta 0), pack-reusados 0
To github.com:NerineDev/proyecto-git-ud6-ddaw.git
* [new branch]      main -> main
```

Figura 17. Envío inicial de la rama main al repositorio remoto con git push.

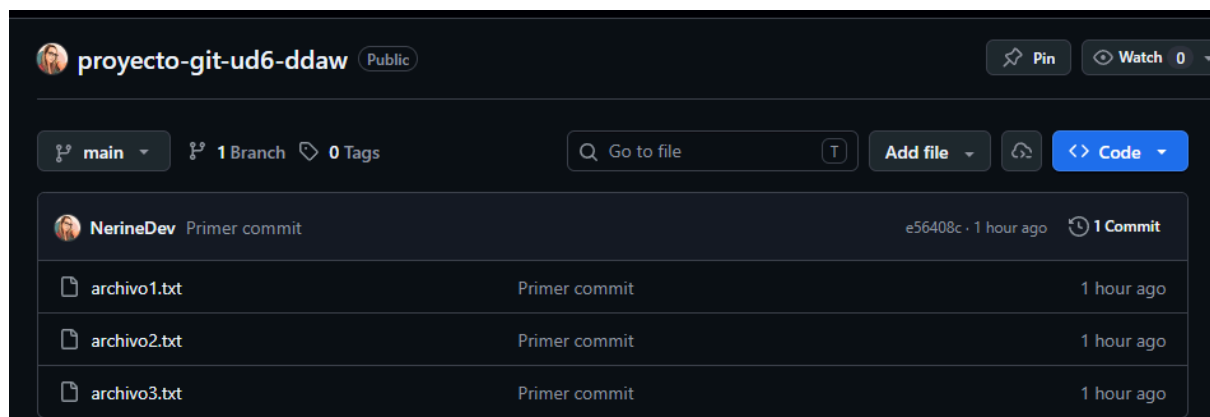


Figura 18. Repositorio remoto en GitHub tras subir los archivos del proyecto.

6. Control de cambios: modificación, commit y actualización remota.

Tras subir el primer estado del proyecto a GitHub, se realizó una modificación en uno de los archivos para comprobar el flujo normal de trabajo con Git. En este caso, se añadió una nueva línea al archivo archivo1.txt mediante el siguiente comando:

```
echo "Cambio de prueba para comprobar el seguimiento de Git" >>
archivo1.txt
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ echo "Cambio de prueba para co
mprobar el seguimiento de Git" >> archivo1.txt
```

Figura 19. Modificación del archivo1.txt

Después de modificar el archivo, se ejecutó el comando:

git status

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios no rastreados para el commit:
  (usa "git add <archivo>..." para actualizar lo que será confirmado)
  (usa "git restore <archivo>..." para descartar los cambios en el directorio de
  trabajo)
modificados:    archivo1.txt
```

Figura 20. Detección de cambios en archivo1.txt mediante git status.

Git detectó que archivo1.txt había cambiado respecto al último commit. Este paso es importante porque permite revisar qué archivos han sido modificados antes de preparar una nueva confirmación.

A continuación, se añadió únicamente el archivo modificado al área de preparación:

git add archivo1.txt

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git add archivo1.txt
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
modificados:    archivo1.txt
```

Figura 21. Actualización del archivo1.txt en el staging area.

Después se comprobó de nuevo el estado del repositorio para verificar que el archivo estaba preparado para el commit. Una vez revisado, se confirmó el cambio con el comando:

git commit -m "Actualiza archivo de prueba"

Este commit creó una nueva versión del proyecto que incluía la modificación realizada en archivo1.txt. Posteriormente, se envió el cambio al repositorio remoto mediante:

git push

```

nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git commit -m "Actualiza archi
vo de prueba"
[main 7fed80d] Actualiza archivo de prueba
 1 file changed, 1 insertion(+)
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git push
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 2 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 394 bytes | 394.00 KiB/s, listo.
Total 3 (delta 0), reusados 0 (delta 0), pack-reusados 0
To github.com:NerineDev/proyecto-git-ud6-ddaw.git
 e56408c..7fed80d  main -> main

```

Figura 22. Push desde Git con la modificación a archivo1.txt

Al estar ya configurado el seguimiento entre la rama local main y la rama remota origin/main, no fue necesario indicar de nuevo el origen ni la rama. Finalmente, se comprobó en GitHub que el archivo archivo1.txt aparecía actualizado con la nueva línea añadida.

 NerineDev	Actualiza archivo de prueba	7fed80d · 2 minutes ago	🕒 2 Commits
	archivo1.txt	Actualiza archivo de prueba	2 minutes ago
	archivo2.txt	Primer commit	1 hour ago
	archivo3.txt	Primer commit	1 hour ago

Figura 23. Archivo archivo1.txt actualizado en GitHub tras realizar git push.

proyecto-git-ud6-ddaw / archivo1.txt		
	NerineDev Actualiza archivo de prueba	
Code	Blame	2 lines (2 loc) · 69 Bytes
1	Archivo test 1	
2	Cambio de prueba para comprobar el seguimiento de Git	

Figura 24. Visualización del archivo1.txt desde GitHub

7. Descarga de cambios remotos mediante git pull.

Además de enviar cambios desde el repositorio local hacia GitHub, también se comprobó el proceso contrario: descargar en Ubuntu una modificación realizada directamente en el repositorio remoto. Para ello, se editó el archivo archivo2.txt desde la interfaz web de GitHub y se añadió una nueva línea de prueba.



Figura 25. Cambio realizado en archivo2.txt desde GitHub para probar la descarga remota.

Después de confirmar el cambio en GitHub, se volvió a la terminal de Ubuntu y se ejecutó el comando:

```
git pull
```

Este comando descarga los cambios existentes en el repositorio remoto y los integra en la rama local. En este caso, Git actualizó la rama main y mostró que el archivo archivo2.txt había recibido una inserción nueva.

Para comprobar que el cambio remoto se había incorporado correctamente al repositorio local, se utilizó:

```
cat archivo2.txt
```

```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Desempaquetando objetos: 100% (3/3), 1.00 KiB | 1.00 MiB/s, listo.
Desde github.com:NerineDev/proyecto-git-ud6-ddaw
 7fed80d..a765a86  main      -> origin/main
Actualizando 7fed80d..a765a86
Fast-forward
 archivo2.txt | 1 +
 1 file changed, 1 insertion(+)
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ cat archivo2.txt
Archivo test 2
Cambio realizado desde GitHub para probar git pull.
```

Figura 26. Descarga del cambio remoto mediante git pull y comprobación del contenido actualizado con cat.

La salida mostró el contenido actualizado del archivo, incluyendo la línea añadida desde GitHub. Finalmente, se revisó el historial mediante:

```
git log --oneline
```

En el historial aparecía el nuevo commit realizado desde GitHub junto con los commits anteriores. De esta forma, se comprobó el funcionamiento completo del flujo remoto-local: realizar un cambio en GitHub, descargarlo con git pull y verificarlo en el entorno local.

```
nerlne@nerlne-VirtualBox:~/proyecto-git-ud6-ddaw$ git log --oneline
a765a86 (HEAD -> main, origin/main) Add test message to archivo2.txt
7fed80d Actualiza archivo de prueba
e56408c Primer commit
```

Figura 27. Historial del repositorio tras incorporar el commit realizado desde GitHub.

8. Accesibilidad y seguridad del repositorio

La accesibilidad y la seguridad del repositorio son aspectos importantes cuando se trabaja con sistemas de control de versiones, especialmente si el proyecto se almacena también en una plataforma remota como GitHub. Un repositorio debe ser accesible para las personas autorizadas, pero al mismo tiempo debe proteger el código y evitar la exposición de información sensible.

En GitHub, la accesibilidad puede gestionarse mediante la visibilidad del repositorio. Un repositorio público puede ser consultado por cualquier usuario, mientras que un repositorio privado solo está disponible para las personas autorizadas. Además, cuando se trabaja en equipo, se pueden asignar distintos permisos a los colaboradores, por ejemplo permisos de lectura, escritura o administración. De esta forma, cada persona puede realizar solo las acciones necesarias según su función dentro del proyecto.

En esta práctica se utilizó GitHub como repositorio remoto y se configuró el acceso mediante clave SSH. Este método permite autenticar el equipo local de forma segura sin introducir la contraseña de la cuenta en cada operación. Para ello, se generó una clave SSH en Ubuntu y se añadió la clave pública a la cuenta de GitHub. La comprobación de conexión mostró que la autenticación se había realizado correctamente, lo que confirma que el equipo quedó autorizado para comunicarse con el repositorio remoto.

También es recomendable activar la autenticación en dos factores en la cuenta de GitHub. Esta medida añade una capa adicional de seguridad, ya que no basta con

conocer la contraseña para acceder a la cuenta. En proyectos colaborativos, otra medida útil es proteger la rama principal, normalmente llamada main, para impedir que se realicen cambios directos sin revisión previa. Así se reduce el riesgo de subir código defectuoso o no revisado al repositorio principal.

Otro elemento importante es el archivo `.gitignore`. Este archivo sirve para indicar a Git qué archivos o carpetas no deben incluirse en el repositorio. Es especialmente útil para evitar subir archivos temporales, dependencias generadas automáticamente o información sensible. Por ejemplo, en un proyecto real no deberían subirse archivos como `.env`, ya que pueden contener contraseñas, claves API o datos privados de configuración.

Un ejemplo básico de contenido para un archivo `.gitignore` sería:

```
.env
node_modules/
dist/
*.log
```

Gracias a estas medidas, el repositorio puede mantenerse accesible para las personas que necesitan trabajar con él, pero protegido frente a accesos indebidos, cambios no controlados o publicación accidental de información sensible.

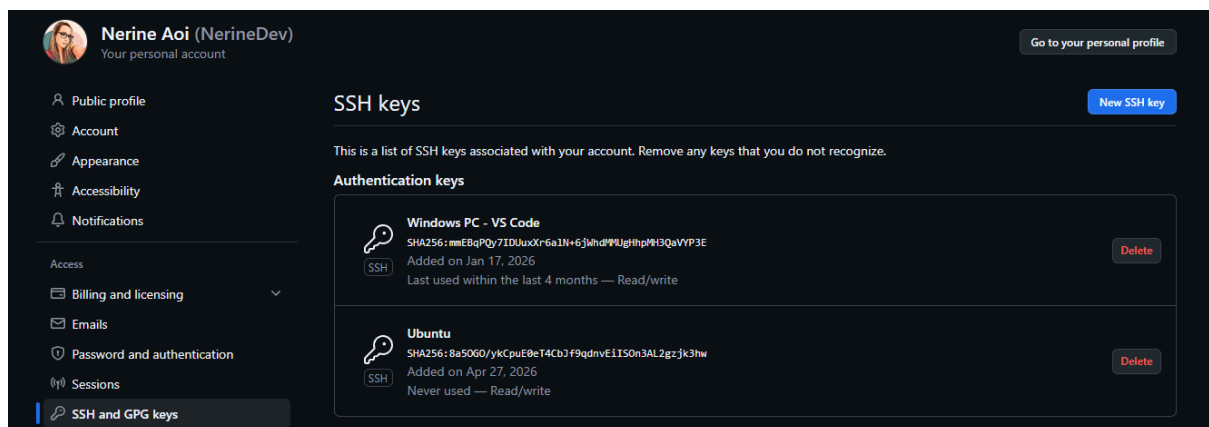


Figura 28. Claves SSH asociadas a la cuenta de GitHub, utilizadas para autenticar el acceso seguro desde Ubuntu.

9. Documentación del sistema de control de versiones

Además de instalar y utilizar Git, es importante documentar correctamente el sistema de control de versiones empleado en el proyecto. Esta documentación permite que cualquier persona que se incorpore al desarrollo pueda entender cómo está organizado el repositorio, qué comandos básicos debe utilizar y qué normas debe seguir para trabajar de forma ordenada.

El primer archivo recomendado en cualquier repositorio es README.md. Este archivo funciona como presentación principal del proyecto y suele incluir el nombre de la aplicación, una descripción breve, los pasos de instalación, instrucciones de uso y cualquier información inicial necesaria para ejecutar o revisar el proyecto.

En esta práctica se añadió un archivo README.md al repositorio para documentar el objetivo del proyecto, los archivos incluidos y los comandos principales utilizados. Gracias a este archivo, cualquier persona que acceda al repositorio puede comprender rápidamente para qué se ha creado, qué pruebas se han realizado y qué operaciones de Git se han utilizado durante la práctica.

Otro archivo importante es .gitignore. Su función es indicar a Git qué archivos o carpetas no deben incluirse en el control de versiones. Esto permite evitar que se suban archivos generados automáticamente, dependencias, configuraciones locales o datos sensibles. En proyectos reales, este archivo ayuda a mantener el repositorio limpio y seguro.

También puede incluirse un archivo CONTRIBUTING.md cuando el proyecto se desarrolla en equipo. Este documento explica cómo deben colaborar los miembros del equipo, cómo crear ramas, cómo escribir mensajes de commit, cómo abrir pull requests y qué normas deben respetarse antes de fusionar cambios en la rama principal.

Por otra parte, el archivo CHANGELOG.md permite registrar los cambios importantes realizados en cada versión del proyecto. Este archivo resulta útil para consultar la evolución del desarrollo y conocer qué funcionalidades, correcciones o modificaciones se han incorporado con el tiempo.

En proyectos más completos, también puede existir una carpeta docs/, destinada a guardar documentación técnica adicional, manuales, diagramas, guías de instalación o explicaciones sobre la arquitectura del sistema. De esta forma, la documentación no queda dispersa y puede mantenerse junto al propio código fuente.

En conjunto, estos archivos ayudan a que el repositorio sea más comprensible, mantenible y seguro. Git permite registrar los cambios, pero la documentación explica cómo debe utilizarse el repositorio y facilita que el trabajo sea más claro tanto para el autor como para otros posibles colaboradores.

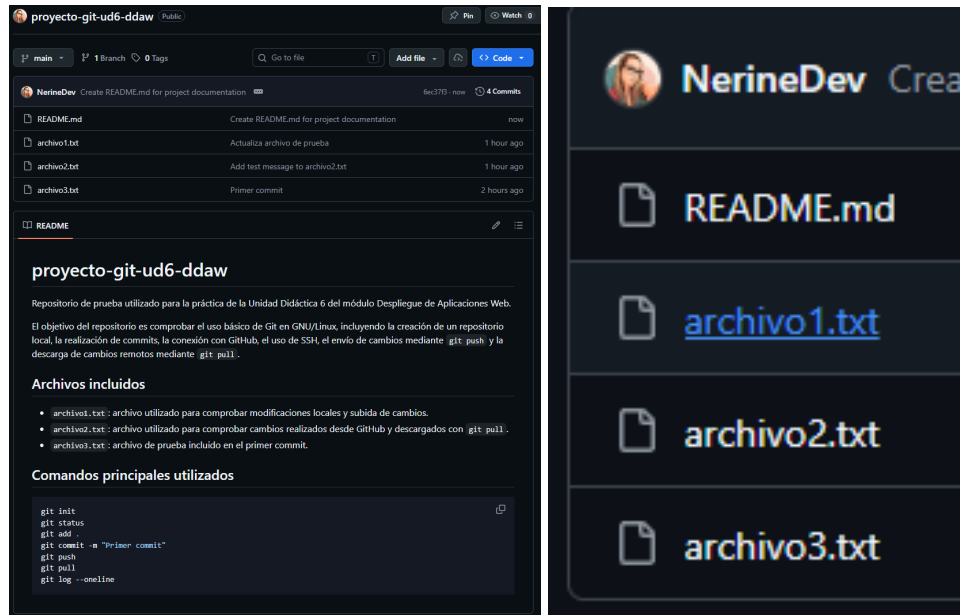


Figura 29. Ejemplo de estructura básica de un repositorio documentado en GitHub.

10. Gestión de flujos de trabajo

La gestión de flujos de trabajo define cómo deben organizarse los cambios dentro de un repositorio para evitar errores, pérdidas de información o modificaciones desordenadas. Aunque en esta práctica se ha trabajado con un repositorio sencillo e individual, el procedimiento seguido representa la base de un flujo de trabajo habitual con Git y GitHub.

El flujo aplicado comenzó con la creación de un repositorio local mediante `git init`. Después se añadieron archivos al área de preparación con `git add` y se confirmaron los cambios mediante `git commit`. Una vez creado el repositorio remoto en GitHub, se configuró la conexión con GitHub mediante SSH y se enviaron los cambios con `git push`. Posteriormente, se comprobó también el proceso inverso realizando una modificación desde GitHub y descargándola en Ubuntu mediante `git pull`.

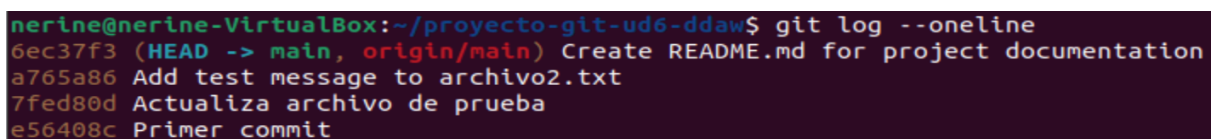
En un proyecto colaborativo, este flujo puede ampliarse mediante el uso de ramas. La rama principal, normalmente `main`, debe reservarse para versiones estables del proyecto. Cuando se quiere añadir una nueva funcionalidad o corregir un error, lo recomendable es crear una rama independiente, trabajar sobre ella y fusionarla con `main` solo cuando los cambios estén revisados.

Un ejemplo básico de comandos para trabajar con una rama sería:

```
git checkout -b nueva-funcionalidad
git add .
git commit -m "Añade nueva funcionalidad"
git checkout main
git merge nueva-funcionalidad
```

Este tipo de organización permite separar el desarrollo de nuevas funciones del código principal. Además, en plataformas como GitHub se puede utilizar el sistema de pull requests, que permite revisar los cambios antes de integrarlos en la rama principal. De esta forma, el equipo puede comentar, corregir y aprobar el código antes de incorporarlo al proyecto estable.

Por tanto, una gestión adecuada del flujo de trabajo con Git ayuda a mantener el repositorio ordenado, facilita la colaboración y reduce el riesgo de errores. Incluso en proyectos pequeños, seguir una secuencia clara de cambios, commits, push y pull permite conservar un historial comprensible y recuperar versiones anteriores si fuera necesario.



```
nerine@nerine-VirtualBox:~/proyecto-git-ud6-ddaw$ git log --oneline
6ec37f3 (HEAD -> main, origin/main) Create README.md for project documentation
a765a86 Add test message to archivo2.txt
7fed80d Actualiza archivo de prueba
e56408c Primer commit
```

Figura 30. Historial de commits utilizado para comprobar la evolución del repositorio durante la práctica.

11. Conclusión

A lo largo de esta práctica se ha instalado, configurado y utilizado Git como sistema de control de versiones en un entorno GNU/Linux. En primer lugar, se realizó la instalación mediante el gestor de paquetes APT de Ubuntu y se comprobó su correcto funcionamiento con el comando `git --version`. Después, se configuró la identidad del usuario para que los commits quedaran correctamente asociados a un nombre y correo electrónico.

También se creó un repositorio local desde cero, se añadieron archivos de prueba y se realizó un primer commit para registrar el estado inicial del proyecto. A partir de ahí, se comprobó el flujo básico de trabajo con Git, incluyendo la modificación de archivos, el uso de `git status`, `git add`, `git commit`, `git push` y `git pull`. Estos comandos permiten controlar los cambios, revisar el estado del repositorio, guardar versiones y sincronizar el trabajo local con un repositorio remoto.

La conexión con GitHub permitió comprobar la utilidad de los repositorios remotos para conservar una copia del proyecto en la nube y facilitar el trabajo desde otros equipos o con otros colaboradores. Además, la configuración mediante clave SSH aportó una forma más segura de autenticación, evitando introducir la contraseña de la cuenta en cada operación.

Por último, se revisó la importancia de documentar correctamente el repositorio mediante archivos como README.md, .gitignore, CONTRIBUTING.md o CHANGELOG.md. En esta práctica se añadió un README.md para explicar el objetivo del repositorio, los archivos incluidos y los comandos principales utilizados. En conjunto, Git no solo permite guardar versiones del código, sino también trabajar de forma más organizada, segura y trazable durante el desarrollo de aplicaciones web.

12. Bibliografía y recursos

- Material de la Unidad Didáctica 6: Documentación, sistemas de control de versiones y de integración continua. Módulo Despliegue de Aplicaciones Web.
- Documentación oficial de Git. “Getting Started - Installing Git”.
<https://git-scm.com/book/en/v2/Getting-Started-Installing-Git>
- Documentación oficial de Git. “First-Time Git Setup”.
<https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup>
- Documentación oficial de GitHub. “Connecting to GitHub with SSH”.
<https://docs.github.com/en/authentication/connecting-to-github-with-ssh>
- Documentación oficial de GitHub. “Adding a new SSH key to your GitHub account”.
<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account>
- Documentación oficial de GitHub. “About READMEs”.
<https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
- The Wild Transistor. “APT en Debian explicado: Instalar, actualizar y limpiar paquetes de forma sencilla.” YouTube.
<https://www.youtube.com/watch?v=PQNYGNW3HcM>

- HolaMundo. “Aprende linux ahora! curso desde cero para principiantes.” YouTube.
<https://www.youtube.com/watch?v=L906Kti3gzE>
- Apuntes correspondientes a la unidad de Despliegue de Aplicaciones Web.
- Curso de Linux en Udemy utilizado como apoyo complementario.